APPENDIX D

The Tactics Used in This Book

A tactic is an instruction that transforms the current goal: it either closes it, or replaces it with simpler goals. Every tactic proof in this book is built from the few dozen tactics described here. Each one has a small example below, taken from `code/lean/Book/AppD/Tactics.lean`, which compiles. For the reasoning behind a tactic, follow the chapter references; for any tactic, hovering over its name in the editor shows its full documentation.

In this appendix you will learn:

- which tactics close goals, and which decision procedures they run;
- how the introduction and elimination rules of natural deduction appear as tactics;
- how to rewrite and simplify, and how to keep `simp` predictable;
- how to structure a longer proof, and how termination is proved;
- which tactics come with Lean and which with Mathlib.

D.1 Reading a goal

Inside a `by` block, the infoview shows the **goal**: the hypotheses, one per line with their names and types, then the turnstile `⊢`, then the proposition still to be proved. A tactic proof is a conversation with this display: every step changes it, and the proof is finished when no goals remain. Chapter 2 introduces the infoview; Chapter 4 explains why the goal is a sequent $\Gamma \vdash A$ of natural deduction, which is why the hypotheses are above the turnstile and the goal below it.

Several goals can be open at once (after `constructor` on `p ∧ q`, for example). Tactics act on the first one; the bullet `·` focuses on one goal until it is closed, and indentation shows the structure.

D.2 Closing goals

```
-- `rfl`: both sides compute to the same value.
example : 2 + 2 = 4 := by rfl

-- `exact e`: the term `e` has exactly the type of the goal.
example (p : Prop) (hp : p) : p := by exact hp

-- `assumption`: some hypothesis has the type of the goal.
example (p q : Prop) (_hq : q) (hp : p) : p := by assumption

-- `trivial`: closes `True`, and goals that `rfl` or `assumption` close.
example : True := by trivial
```

```
-- `contradiction`: the hypotheses are contradictory (here, `False` itself).
example (p : Prop) (h : False) : p := by contradiction

-- `absurd h h'`: `h : p` and `h' : ¬p` prove anything.
example (p q : Prop) (hp : p) (hnp : ¬p) : q := absurd hp hnp

-- `decide`: evaluate a decidable proposition.
example : 7 ∣ 91 := by decide

-- `omega`: linear arithmetic over ℕ and ℤ.
example (a b : ℕ) (h : a < b) : a + 1 ≤ b := by omega

-- `ring`: identities in commutative (semi)rings.
example (x y : ℤ) : (x - y) * (x + y) = x ^ 2 - y ^ 2 := by ring

-- `linarith`: linear inequalities over ordered fields (and ℕ, ℤ).
example (x y : ℚ) (h₁ : x ≤ 2 * y) (h₂ : y < 3) : x < 6 := by linarith

-- `norm_num`: numeric facts.
example : (2 : ℚ) / 3 + 1 / 3 = 1 := by norm_num

-- `positivity`: goals of the form `0 < e` or `0 ≤ e`.
example (x : ℤ) : 0 ≤ x ^ 2 + 1 := by positivity

-- `tauto`: propositional tautologies (classically).
example (p q : Prop) : p ∧ q → q ∨ p := by tauto
```

Tactic	Closes the goal when…
`rfl`	both sides of `=` (or `↔`) are equal by computation
`exact e`	the term `e` has the goal's type
`assumption`	a hypothesis has the goal's type
`trivial`	the goal is `True`, or `rfl` / `assumption` works
`contradiction`	the hypotheses are evidently contradictory
`decide`	the proposition is decidable and evaluates to `true`
`omega`	the goal is linear arithmetic over `ℕ` or `ℤ`
`ring`	the goal is an identity of commutative (semi)rings
`linarith`	the goal follows linearly from hypotheses that are inequalities
`norm_num`	the goal is a numerical fact
`positivity`	the goal is `0 < e` or `0 ≤ e` for a visibly positive `e`
`tauto`	the goal is a propositional tautology (reasoning classically)
`grind`	a combination of rewriting and decision procedures succeeds

Chapter 1 uses `rfl`, `decide`, `omega`, `ring` and `grind` for the first time, and Chapter 2 introduces the arithmetic tactics that come with Mathlib. `decide`, `omega`, `ring`, `linarith` and `norm_num` are **decision procedures**: algorithms that are guaranteed to find a proof for every goal in their class. Use them freely once you understand what they decide. They also produce real proof terms, which the kernel checks; but note that `decide` on a large search space can be slow, and `native_decide` (which this book avoids) trusts compiled code instead of the kernel. `grind` and `tauto` reason classically, so proofs that use them depend on `Classical.choice`, as `#print axioms` shows (Chapters 1 and 5).

D.3 Introduction rules: working on the goal

```
-- `intro`: the →I and ∀I rules; name the hypothesis or variable.
example (p q : Prop) : p → q → p := by
  intro hp _hq
  exact hp

-- `constructor`: apply the (only) constructor: ∧I, ↔I, building a structure.
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  · exact hp
  · exact hq

-- `left` / `right`: ∨I, choosing the side.
example (p q : Prop) (hq : q) : p ∨ q := by
  right
  exact hq

-- `exists` / `use`: ∃I, giving the witness.
example : ∃ n : ℕ, n * n = 49 := by
  exists 7

example : ∃ n : ℕ, n + 3 = 7 := by
  use 4

-- `apply f`: backward reasoning; the remaining premises of `f` become goals.
example (p q r : Prop) (hpq : p → q) (hqr : q → r) (hp : p) : r := by
  apply hqr
  apply hpq
  exact hp

-- `refine`: like `exact`, with `?_` holes that become new goals.
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  refine ⟨?_, hq⟩
  exact hp

-- `exfalso`: replace the goal by `False` (⊥E, read backwards).
example (p : Prop) (h : 1 = 0) : p := by
  exfalso
  omega

-- `funext`: two functions are equal if they agree everywhere.
example : (fun n : ℕ => n + 0) = (fun n => n) := by
  funext n
  rfl
```

Each of these tactics applies an *introduction* rule backwards: it looks at the shape of the goal and reduces it to what that rule requires (Chapter 4).

Goal	Tactic	Rule (Chapter 4)
`p → q`, `∀ x, P x`	`intro h`, `intro x`	→I, ∀I
`p ∧ q`, `p ↔ q`, a structure	`constructor`	∧I, ↔I
`p ∨ q`	`left`, `right`	∨I
`∃ x, P x`	`exists w`, `use w`	∃I
`False`-free goal you want to refute	`exfalso`	⊥E, read backwards
`f = g` for functions	`funext x`	function extensionality
anything, using a lemma `f : A → B → goal`	`apply f`, `refine f ?_ ?_`	→E, read backwards

`apply` and `refine` are the general tools: `apply f` unifies the conclusion of `f` with the goal and leaves its remaining premises as new goals; `refine e` does the same for a term `e` with explicit holes `?_`.

D.4 Elimination rules: working on hypotheses

```
-- `cases`: one goal per constructor of the hypothesis (∨E, ∧E, ∃E, pattern matching).
example (p q : Prop) (h : p ∨ q) : q ∨ p := by
  cases h with
  | inl hp => exact Or.inr hp
  | inr hq => exact Or.inl hq

-- `obtain` / `rcases`: take a hypothesis apart with a pattern, in one step.
example (p q r : Prop) (h : p ∧ (q ∧ r)) : r := by
  obtain ⟨_hp, _hq, hr⟩ := h
  exact hr

example (n : ℕ) (h : ∃ k, n = 2 * k) : n % 2 = 0 := by
  rcases h with ⟨k, rfl⟩
  omega

-- `rintro`: `intro` followed by `rcases`.
example (p q : Prop) : p ∧ q → q := by
  rintro ⟨_, hq⟩
  exact hq

-- `induction`: structural induction; one goal per constructor, with hypotheses.
theorem sum_formula (n : ℕ) : 2 * (List.range (n + 1)).sum = n * (n + 1) := by
  induction n with
  | zero => rfl
  | succ k ih =>
    rw [List.range_succ, List.sum_append, mul_add, ih]
    simp
    ring

-- `by_cases`: split on whether a proposition holds.
example (n : ℕ) : n % 2 = 0 ∨ n % 2 = 1 := by
  by_cases h : n % 2 = 0
  · exact Or.inl h
  · right
    omega

-- `split`: one goal per branch of an `if` or `match` in the goal.
example (n : ℕ) : (if n < 5 then n else 5) ≤ 5 := by
  split
  · omega
  · omega

-- `subst`: replace a variable everywhere using an equation `x = e`.
example (a b : ℕ) (h : a = b + 1) : a - 1 = b := by
  subst h
  omega

-- `nomatch`: a hypothesis with no possible constructor proves anything.
example (h : Nat.succ 3 = 0) : False := nomatch h

-- `injection`: constructors are injective.
example (a b : ℕ) (h : Nat.succ a = Nat.succ b) : a = b := by
  injection h
```

Hypothesis	Tactic	Rule
`h : p ∨ q`	`cases h`, `rcases h with hp | hq`	∨E (case analysis)
`h : p ∧ q`, `h : ∃ x, P x`	`obtain ⟨hp, hq⟩ := h`	∧E, ∃E
a value of an inductive type	`cases x`, `induction x`	case analysis, induction
`h : x = e`	`subst h`, `rw [h]`	equality elimination
`h` with no possible constructor	`nomatch h`, `cases h`	⊥E
`h : C a = C b`	`injection h`	constructors are injective
any proposition `p`	`by_cases h : p`	excluded middle (decidable `p`)
`if` / `match` in the goal	`split`	case analysis on the condition

`rcases`, `obtain` and `rintro` take *patterns*: `⟨a, b⟩` takes a pair or an existential apart, `a | b` splits a disjunction, and `rfl` substitutes an equation on the spot. `induction ... with` names each case and its induction hypotheses; adding `generalizing x` keeps `x` universally quantified in the hypothesis, which many proofs need (Chapter 31). When a case introduces names that Lean generated itself (they print with a dagger, `a✝`), `rename_i` gives them names you can use.

D.5 Rewriting and simplification

```
-- `rw [h]`: replace the left side of the equation `h` by its right side.
example (a b c : ℕ) (h₁ : a = b) (h₂ : b = c) : a = c := by
  rw [h₁, h₂]

-- `rw [← h]`: rewrite right to left.
example (a b : ℕ) (h : a = b) (p : ℕ → Prop) (hb : p b) : p a := by
  rw [h]
  exact hb

-- `simp`: rewrite with the simp lemmas (and the ones given) until nothing changes.
example (xs : List ℕ) : (xs ++ []).length = xs.length := by simp

-- `simp only [...]`: only the given lemmas and definitions (predictable).
def double (n : ℕ) : ℕ := n + n

example (n : ℕ) : double n = 2 * n := by
  simp only [double]
  omega

-- `simp at h`, `simp_all`: simplify hypotheses too.
example (a : ℕ) (h : a + 0 = 3) : a = 3 := by
  simp at h
  exact h

-- `simpa using e`: simplify the goal and `e`, then close the goal with `e`.
example (a : ℕ) (h : 0 + a = 3) : a = 3 := by simpa using h

-- `unfold`: unfold a definition.
example (n : ℕ) : double n = n + n := by
  unfold double
  rfl

-- `calc`: a chain of steps, each with its own proof.
example (a b c : ℕ) (h₁ : a ≤ b) (h₂ : b < c) : a < c + 1 :=
  calc a ≤ b := h₁
    _ < c := h₂
    _ < c + 1 := Nat.lt_succ_self c
```

```
-- `symm`, `congr`: symmetry of equality; equal arguments give equal results.
example (a b : ℕ) (h : a = b) : b = a := by
  symm
  exact h

example (f : ℕ → ℕ) (a b : ℕ) (h : a = b) : f a = f b := by
  congr
```

`rw [h]` rewrites with one equation at a time, left to right (`← h` for right to left), and fails if the left side does not occur. `simp` rewrites with a whole database of lemmas tagged `@[simp]`, plus the ones you list, until nothing changes; it is powerful but can be unpredictable, and a `simp` that closes a goal today may stop working when the library changes. Two habits keep proofs robust:

- prefer `simp only [lemma₁, lemma₂, definition]` in the middle of a proof, so that exactly the listed rewrites happen;
- use plain `simp` (or `simpa`) to *finish* a goal, where it does not matter how it gets there.

`simp at h` simplifies a hypothesis, and `simp_all` simplifies everything with everything. `calc` writes a chain of equalities and inequalities as it would appear on paper, each step with its own justification; it is often the clearest way to present an argument.

D.6 Structuring a proof

```
-- `have`: prove an intermediate fact and add it to the context.
example (a b : ℕ) (h : a < b) : a < b + b := by
  have hb : b ≤ b + b := Nat.le_add_right b b
  omega

-- `show`: state the goal (in a form that is equal by computation).
example (n : ℕ) : double n = n + n := by
  show n + n = n + n
  rfl

-- `case`, `next` and `·`: focus on one goal at a time.
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  case left => exact hp
  case right => exact hq

-- `all_goals tac` runs `tac` on every goal; `t₁ <;> t₂` runs `t₂` on every
-- goal that `t₁` produces; `first | t₁ | t₂` tries `t₁`, then `t₂`.
example (p : Prop) (hp : p) : p ∧ p ∧ p := by
  refine ⟨?_, ?_, ?_⟩
  all_goals exact hp

example (n : ℕ) : n + 0 = n ∧ 0 + n = n := by
  constructor <;> simp

example (a b : ℕ) (h : a = b) : b = a := by
  first
  | rfl
  | exact h.symm
```

- `have h : P := proof` (or `have h : P := by tactics`) proves an intermediate fact. Most readable proofs are a short sequence of `have`s followed by a finishing tactic.
- `show P` restates the goal in an equivalent form; it documents the proof and can unfold definitions.

- `case name => ...` and `next => ...` focus on a named or the next goal; the bullet `·` focuses on the first goal.
- `all_goals t`, `t₁ <;> t₂`, `first | t₁ | t₂` and `repeat t` combine tactics when several goals need the same treatment.

D.7 Recursion and termination

```
-- `termination_by`: the measure that decreases in every recursive call.
def log2 (n : ℕ) : ℕ :=
  if h : n < 2 then 0 else 1 + log2 (n / 2)
termination_by n
decreasing_by omega

/-- info: 10 -/
#guard_msgs in
#eval log2 1024
```

Every definition must terminate (Chapter 2). Structural recursion is checked automatically; for other recursion, `termination_by` names a measure that decreases, and `decreasing_by` proves that it does, usually with `omega` or `simp`. When Lean cannot find the measure by itself, the error message shows the goal it could not prove (Chapters 7 and 31). A function defined this way can be reasoned about with `fun_induction` or with the induction principle Lean generates for it (`f.induct`), whose cases follow the function's own recursion.

D.8 Core Lean and Mathlib

All the tactics above come with Lean itself except these, which come from Mathlib (and need an `import`, for example `import Mathlib.Tactic.Ring`):

Tactic	Import
`ring`	`Mathlib.Tactic.Ring`
`linarith`	`Mathlib.Tactic.Linarith`
`norm_num`	`Mathlib.Tactic.NormNum`
`positivity`	`Mathlib.Tactic.Positivity`
`tauto`	`Mathlib.Tactic.Tauto`
`use`	`Mathlib.Tactic.Use`

D.9 Searching and debugging

- `exact?` searches the library for a lemma that closes the goal and prints the call it found; `apply?` lists lemmas that could be applied. Replace them with the suggestion once found.
- `trace_state` prints the current goal in the messages, which is useful in a terminal without an infoview.
- `sorry` closes any goal without a proof. It is how you sketch a proof top-down, but Lean warns about every use, and `#print axioms` reports `sorryAx` for any theorem that depends on one. No proof in this book uses it; Chapter 2 shows the warning.